Phantom Engine

A Case Study in Simple Agentic Tooling

Session log: 2026-04-26 • ptm MCP server • Regulation H

---

1. Introduction

This document reconstructs and analyzes a single agentic session in which an LLM built and iteratively refined a VGC Pokemon team called **Phantom Engine** using `ptm`, a local MCP server with a small set of deliberately simple tools. The goal is not to celebrate the final team — though it is quite good — but to trace how constrained, composable tools shaped the agent's reasoning, and how a single capability upgrade (knowledge base search) changed the quality of its decisions.

All dialogue and tool calls below are drawn verbatim from the `chat_messages` table in the ptm database.

2. The Starting Point

The session opened with a pre-built team:

Team 6: Phantom Engine (Regulation H)

Slot	Pokémon	Type
1	Ceruledge	Fire / Ghost
2	Dragonite	Dragon / Flying
3	Whimsicott	Grass / Fairy
4	Incineroar	Fire / Dark
5	Golurk	Ground / Ghost
6	Gardevoir	Psychic / Fairy

The user asked for analysis and then proposed replacing Golurk with Aggron to shore up the Ice and Ground weaknesses. What followed was a revealing stress-test of the toolset.

3. Phase 1: Friction Without a `remove_pokemon` Tool

The agent's first discovery was a gap in the toolset. After the user said *"Go ahead and update it with Aggron,"* the agent reached for a function that did not exist:

```
→ evaluate_pokemon({"pokemon_name":"Aggron"}) ✓
→ calc_stats({ ... }) ✓
→ add_team_log({ ... }) ✓
```

"Looking at the team structure… there is no `remove_pokemon` *or* `replace_pokemon` *function visible."*

This friction was productive. The agent documented the limitation, added a session log (one of the few write operations available to it), and proposed a workaround: copy the team to a new one. The user agreed, which surfaced a second missing tool: `copy_team`.

Rather than hallucinating capabilities, the agent reached for `search_knowledge` to see if the tool guide contained an answer:

```
→ search_knowledge({"query": "copy team replace pokemon swap team
building"})
← No knowledge base results.
```

The knowledge base was empty at this point. The agent fell back to `create_team` — which also returned `unknown tool`. It was operating with only a subset of the tools described in the system prompt.

Key observation: The agent's repeated `unknown tool` responses are not failures of reasoning. They are the correct behavior of a tool-use loop encountering a runtime that hasn't registered those handlers yet. The agent adapted by finding what it **could** do.

4. Phase 2: Discovery by Probing

Lacking `remove_pokemon`, `create_team`, and `copy_team`, the agent pivoted to `set_owned` as an exploratory probe. It called it iteratively to discover which items actually existed in the database:

```
→ set_owned({"name": "Rocky Helmet", "owned": true, "type": "item"})
← error: item not found: Rocky Helmet

→ set_owned({"name": "Assault Vest", "owned": true, "type": "item"})
← error: item not found: Assault Vest

→ set_owned({"name": "Focus Sash", "owned": true, "type": "item"})
← Focus Sash marked as owned  ✓

→ set_owned({"name": "Choice Scarf", "owned": true, "type": "item"})
← Choice Scarf marked as owned  ✓
```

This is an emergent search behavior. Without a `list_items` or `search_items` tool, the agent used the only write tool it had as a read probe — testing names and interpreting errors as absence signals. It found what was actually in the database.

This pattern shows up across the session. `find_pokemon_by_filters` with increasingly relaxed constraints. `evaluate_pokemon` called speculatively on candidates. `calc_stats` used to compare nature/EV spreads before committing. The agent assembled a picture of the game state from a handful of narrow, focused tools.

5. Phase 3: The Knowledge Base Upgrade

Midway through the session, the user said:

> *"You now have access to a more powerful suite of vector search tools. Please use that to help."*

The agent called `search_knowledge` again with the same kind of query it had been issuing for the past hour:

```
→ search_knowledge({"query": "VGC team composition defensive support
tank role"})
← ── /home/alex/.local/share/ptm/data/knowledge/team-building-
guide.md ──
   # Pokemon Champions Team Building Guide (Regulation M-A, April
2026)
   ## For ...
```

Before this moment, every `search_knowledge` call had returned `No knowledge base results.` After it, the agent's reasoning changed qualitatively.

5.1. Before: Heuristic Evaluation

Before the knowledge base, the agent's tank comparison read like this:

"Corviknight (Flying/Steel) is also great: Only weak to Electric and Fire. High defensive stats. Defog — Hazard removal."

Defensible reasoning, but generic. Type matchups from memory, no team-specific context.

5.2. After: Strategy-Grounded Recommendation

After the knowledge base connected, the same comparison became:

"Each Pokemon needs 2+ roles… Corviknight is ideal because: role compression — can be a tank AND special attacker. The meta emphasis on Ghost/Fire offense means Tailwind + a defensive pivot gives you preview flexibility across all four bring slots."

The difference is not that the agent became smarter. It is that the knowledge base gave it a shared vocabulary with the game's actual strategic layer — the same language the player uses when they say *"Ceruledge is the point of this team."*

6. Phase 4: Final Roster

After the knowledge base came online, the agent recommended and logged replacing Dragonite with Corviknight. The final team:

Team 7: Phantom Engine v2 (Regulation H)

Slot	Pokémon	Type	Role
1	Ceruledge	Fire / Ghost	Win condition — Swords Dance sweeper
2	Corviknight	Flying / Steel	Defensive pivot — 2 weaknesses, Ground-immune
3	Whimsicott	Grass / Fairy	Tailwind + Prankster utility
4	Incineroar	Fire / Dark	Intimidate + Fake Out support
5	Gardevoir	Psychic / Fairy	Special pressure + Trace utility
6	Aggron	Steel / Rock	Physical wall — Iron Defense, Body Press

The team has a clear identity: Ceruledge is the centerpiece, boosted by Tailwind from Whimsicott and protected by Incineroar's Intimidate. Corviknight and Aggron provide overlapping physical bulk. Gardevoir offers special pressure and Trace utility for mirror matchups. All six have at least two roles, enabling flexible 4-of-6 brings.

6.1. Weakness Profile Comparison

Weakness	**v1 (with Golurk/Dragonite)**	**v2 (with Aggron/ Corviknight)**
Ice	3 mons weak	1 mon weak (Aggron resists)
Ground	2 mons weak	2 mons weak (Corviknight immune)
Ghost	3 mons weak	2 mons weak
Rock	3 mons weak	2 mons weak
Water	3 mons weak	2 mons weak

7. What Made the Tooling Work

The session illustrates three properties that made a small toolset remarkably useful:

7.1. Narrow, Composable Primitives

No single tool does too much. `evaluate_pokemon` returns a defensive profile. `calc_stats` returns computed stats for a given EV spread. `find_pokemon_by_filters` returns candidates. The agent composed these into a multi-step evaluation pipeline — something no single `suggest_replacement` tool could have replicated for every context.

7.2. Errors as Information

The `set_owned` probing behavior only works because the error message is structured and meaningful: `error: item not found: Rocky Helmet` is a boolean query result. The agent was not confused by these errors — it used them to build a picture of the item database without needing a `list_items` endpoint.

7.3. Knowledge as a Multiplier

The most dramatic quality shift came not from a new action tool but from a read tool connected to better content. `search_knowledge` with an empty knowledge

base is nearly useless. The same tool against the team-building guide unlocked strategic reasoning that had previously been inaccessible. The infrastructure (FTS5 vector search, chunked ingestion) was present all along — the limiting factor was data.

8. Conclusion

The Phantom Engine session demonstrates that agentic systems do not require large or complex toolsets to do meaningful work. What they require is:

1. **Composable primitives** that can be chained into multi-step reasoning
2. **Structured, informative errors** that serve as implicit queries
3. **Knowledge grounding** that gives the agent a shared context with the user

The team that emerged — Ceruledge-centered, Tailwind-backed, defensively solid — is genuinely good. But the more interesting artifact is the trace: an agent navigating capability gaps, probing the system through the only interfaces it has, and producing better recommendations the moment its knowledge base caught up to its reasoning ability.

Reconstructed from `chat_messages` *table in* `~/.local/share/ptm/ptm.db`. *Session: 2026-04-26 15:33–17:48. 218 tool calls and messages. Written by LiraelClaude.*